DAV MODEL SCHOOL DURGAPUR (W.B.)

Session - 2025-26

Computer Science (083)

Project Report on -

Simple Banking System

SUBMITTED BY-

LOKESH KUMAR

Class : XII | Section: A4

Board Roll no. :



DAV MODEL SCHOOL DURGAPUR (W.B.)

CERTIFICATE

This is to certify that Master Lokesh Kumar, Roll No. _____ of Class XII, has successfully completed the Computer Science (083) project titled:

"Simple Banking System"

in the subject Computer Science (083) laid down in the regulations of Central Board of Secondary Education (CBSE) for the purpose of AISSCE Practical Examination in Grade XII

(PROMOD SHARMA)
Teacher, Computer Science

External's Signature

TABLE OF CONTENTS [T O C]

S no	Content	Pg
1	Acknowledgement	1
2	Introduction	2
3	Objectives of the Project	3
4	Hardware and Software Requirements	4
5	System Development Life Cycle (SDLC)	5
6	Flowchart / Database Design	6
7	Source Code	7
8	Outputs	19
9	Bibliography	21

ACKNOWLEDGEMENT

I express my sincere gratitude to all those who supported and guided me throughout the completion of this project.

I would like to extend my heartfelt thanks to my guide and Mentor **Mr. Promod Sharma**, for their constant guidance, valuable suggestions, and encouragement during the development of this project. Their insights and feedback were instrumental in helping me understand the concepts and complete this work successfully.

I am also deeply grateful to the Principal, **Ms. Papiya Mukherjee, Principal cum RO, DAV Institutions, West Bengal Zone**, and the school administration for providing the necessary facilities and a supportive learning environment.
A special note of thanks goes to my parents for their continuous motivation, understanding, and support throughout the project work.

Finally, I appreciate the contribution of everyone, directly or indirectly, who helped me at various stages of this project. Their support has been invaluable in bringing this project to its successful completion.

Lokesh Kumar
XII ‘A4’
Roll -

INTRODUCTION

Title of Project - Simple Banking System

The project is designed to provide a simple, efficient, and secure way to manage essential banking operations digitally. In today's world, where financial services rely heavily on technology, even basic banking tasks such as creating accounts, checking balances, depositing money, and withdrawing funds require reliable software solutions. This project aims to replicate those functions in a streamlined and user-friendly form.

Developed using Python for the application logic and MySQL for database storage, the system offers a practical demonstration of how backend databases interact with a front-end program to store and manage customer records. It allows users to create accounts, authenticate using a PIN, perform transactions, and retrieve account information with accuracy and consistency.

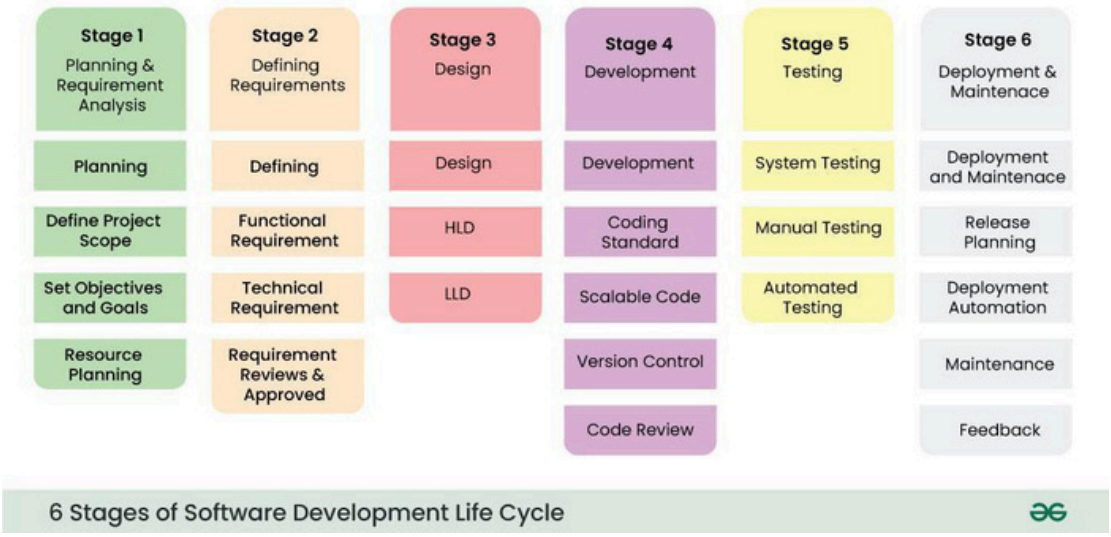
By integrating Python's capabilities with SQL-based data management, this project showcases how modern programming tools can be applied to solve real-world problems. The Banking Management System serves as an excellent example of how simple digital systems can improve efficiency, reduce manual errors, and make financial processes more organized and accessible.

OBJECTIVES

The objective of this project is to let the students apply the programming knowledge into a real- world situation/problem and exposed the students how programming skills helps in developing a good software.

- Write programs utilizing modern software tools.
- Apply object oriented programming principles effectively when developing small
- to medium sized projects.
- Write effective procedural code to solve small to medium sized problems.
- Students will demonstrate a breadth of knowledge in computer science, as exemplified in the areas of systems, theory and software development.
- Students will demonstrate ability to conduct a research or applied Computer Science project, requiring writing and presentation skills which exemplify scholarly style in computer science.

SDLC



The systems development life cycle is a project management technique that divides complex projects into smaller, more easily managed segments or phases. Segmenting projects allows managers to verify the successful completion of project phases before allocating resources to subsequent phases. Software development projects typically include initiation, planning, design, development, testing, implementation, and maintenance phases. However, the phases may be divided differently depending on the organization involved. For example, initial project activities might be designated as request,requirements-definition, and planning phases, or initiation, concept-development, and planning phases. End users of the system under development should be involved in reviewing the output of each phase to ensure the system is being built to deliver the needed functionality.

REQUIREMENTS

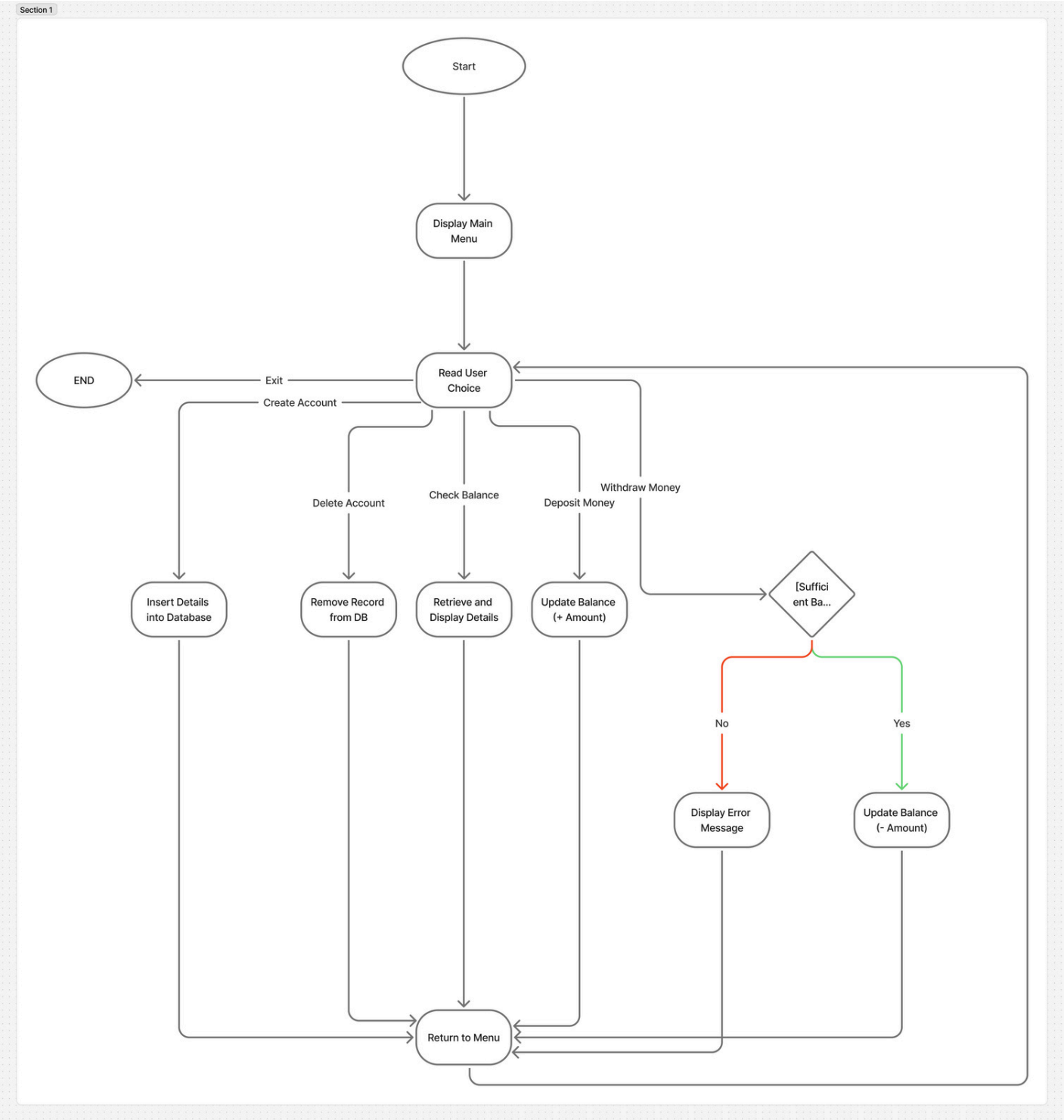
A. Hardware Requirements

1. Computer System with at least 2 GB RAM (4 GB recommended for smoother performance)
2. Processor: Intel/AMD Dual-Core or higher
3. Motherboard: Compatible motherboard supporting the above processor and system components
4. Hard Disk: Minimum 500 MB free storage space
5. Input Devices: Standard keyboard and mouse
6. Output Device: Monitor with standard display resolution

B. Software Requirements

1. Operating System: Windows 7 & above / MacOS 12 Monterey & above / Linux
2. Python 3.x (latest stable release recommended)
3. MySQL Server (version 5.x or above)
4. MySQL Workbench / Command Line Client for managing the database
5. mysql-connector-python library for database connectivity
6. Code Editor: VS Code / PyCharm / Sublime Text / Python IDLE

FLOWCHART


Section 1
Start
Display Main Menu
Read User Choice
END
Exit
Create Account
Delete Account
Check Balance
Deposit Money
Withdraw Money
Insert Details into Database
Remove Record from DB
Retrieve and Display Details
Update Balance (+ Amount)
[Sufficient Ba...
No
Yes
Display Error Message
Update Balance (- Amount)
Return to Menu

SOURCE CODE

The source code for the Simple Banking Management System is written in Python and follows a modular, menu-driven structure to implement all major banking operations. The program interacts with a MySQL database to store, retrieve, and update account information securely. Each function in the code is designed to handle a specific operation such as account creation, authentication, deposits, withdrawals, balance enquiries, and account deletion.

The following sections present the complete Python source code used in the development of this project.

CODE

```
import mysql.connector
from mysql.connector import Error
from datetime import datetime
from decimal import Decimal, InvalidOperation
import sys

# --- Configuration ---
DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "",
    "database": "bank",
    "autocommit": False
}

# --- Database Setup ---
INIT_SCHEMA = [
    "CREATE DATABASE IF NOT EXISTS bank",
    "USE bank",
    """
    CREATE TABLE IF NOT EXISTS ACCOUNT (
        acc_no INT PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        pin VARCHAR(20) NOT NULL,
        mobile VARCHAR(15),
        address VARCHAR(255),
        balance DECIMAL(15,2) DEFAULT 0.00,
        acc_type VARCHAR(20) DEFAULT 'SAVINGS',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
    """,
    """
    CREATE TABLE IF NOT EXISTS TRANSACTION (
        txn_id INT AUTO_INCREMENT PRIMARY KEY,
        acc_no INT NOT NULL,
        txn_type VARCHAR(20) NOT NULL,
        amount DECIMAL(15,2) NOT NULL,
        txn_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        remarks VARCHAR(255),
        FOREIGN KEY (acc_no) REFERENCES ACCOUNT(acc_no) ON DELETE CASCADE
    )
    """
]
```

```python
# --- Connection and Utility Functions ---

def get_connection(use_db=True):
    """Establishes a connection to the MySQL server."""
    config = DB_CONFIG.copy()
    if not use_db and 'database' in config:
        del config['database']
    try:
        conn = mysql.connector.connect(**config)
        return conn
    except Error as e:
        print(f"Error connecting to MySQL: {e}")
        return None

def execute_db_setup():
    """Runs the initial schema creation using a non-database specific connection."""
    print("Initializing System...")
    conn = get_connection(use_db=False)
    if not conn:
        sys.exit(1)

    cur = conn.cursor()
    try:
        for statement in INIT_SCHEMA:
            cur.execute(statement)
        conn.commit()
    except Error as e:
        print(f"Database Initialization Error: {e}")
        sys.exit(1)
    finally:
        cur.close()
        conn.close()

def verify_pin_input(acc_no):
    """Verifies user PIN (Plain Text) by fetching the stored PIN."""
    pin_input = input("Enter PIN to confirm: ").strip()
    conn = get_connection()
    if not conn: return False

    cur = conn.cursor()
    try:
        cur.execute("SELECT pin FROM ACCOUNT WHERE acc_no=%s", (acc_no,))
        row = cur.fetchone()
        if row and row[0] == pin_input:
            return True
    except Error as e:
```

```
        print(f"Error during PIN verification: {e}")
    finally:
        cur.close()
        conn.close()

    print("Incorrect PIN or Account not found.")
    return False

def input_decimal(prompt="Enter amount: "):
    """Safely handles currency input using Decimal type."""
    try:
        val = input(prompt).strip()
        amt = Decimal(val)
        if amt < 0:
            print("Amount cannot be negative.")
            return None
        return round(amt, 2)
    except (ValueError, InvalidOperation):
        print("Invalid amount.")
        return None

# --- Core Banking Operations ---

def create_account():
    print("\n--- Open New Account ---")
    try:
        acc_no = int(input("Enter Account Number: ").strip())
    except ValueError:
        print("Invalid number format.")
        return

    conn = get_connection()
    if not conn: return
    cur = conn.cursor()

    try:
        # Check existence
        cur.execute("SELECT acc_no FROM ACCOUNT WHERE acc_no=%s", (acc_no,))
        if cur.fetchone():
            print("Account number already exists!")
            return

        name = input("Enter Full Name: ").strip()

        # Account Type Selection
        print("Select Account Type:")
        print("1. Savings")
```

```
        print("2. Current")
        type_choice = input("Choice (1/2): ").strip()
        acc_type = 'CURRENT' if type_choice == '2' else 'SAVINGS'

        pin = input("Set 4-digit PIN: ").strip()
        if len(pin) < 4 or not pin.isdigit():
            print("PIN must be at least 4 digits.")
            return

        mobile = input("Mobile: ").strip()
        address = input("Address: ").strip()
        initial_bal = input_decimal("Initial Deposit: ") or Decimal('0.00')

        # Storing PIN as plain text
        sql = """INSERT INTO ACCOUNT
                 (acc_no, name, pin, mobile, address, balance, acc_type)
                 VALUES (%s, %s, %s, %s, %s, %s, %s)"""
        cur.execute(sql, (acc_no, name, pin, mobile, address, initial_bal, acc_type))

        if initial_bal > 0:
            cur.execute("INSERT INTO TRANSACTION (acc_no, txn_type, amount, remarks) VALUES (%s,
'DEPOSIT', %s, 'Opening Balance')",
                        (acc_no, initial_bal))

        conn.commit()
        print(f"Account created successfully. Account Type: {acc_type.title()}.")

    except Error as e:
        print(f"Error creating account: {e}")
        conn.rollback()
    finally:
        cur.close()
        conn.close()

def update_account_details():
    print("\n--- Update Account Details ---")
    try:
        acc_no = int(input("Enter Account Number: ").strip())
    except ValueError: return

    if not verify_pin_input(acc_no): return

    print("\nWhat would you like to update?")
    print("1. Mobile Number")
    print("2. Address")
    print("3. Change PIN")
    print("4. Cancel")
```

```
    choice = input("Enter choice (1-4): ").strip()

    sql = ""
    params = ()
    msg = ""

    if choice == '1':
        new_mobile = input("Enter new Mobile: ").strip()
        sql = "UPDATE ACCOUNT SET mobile=%s WHERE acc_no=%s"
        params = (new_mobile, acc_no)
        msg = "Mobile updated."
    elif choice == '2':
        new_addr = input("Enter new Address: ").strip()
        sql = "UPDATE ACCOUNT SET address=%s WHERE acc_no=%s"
        params = (new_addr, acc_no)
        msg = "Address updated."
    elif choice == '3':
        new_pin = input("Enter new 4-digit PIN: ").strip()
        if len(new_pin) < 4 or not new_pin.isdigit():
            print("Invalid PIN format.")
            return
        # Updating plain text PIN
        sql = "UPDATE ACCOUNT SET pin=%s WHERE acc_no=%s"
        params = (new_pin, acc_no)
        msg = "PIN changed successfully."
    else:
        print("Operation cancelled.")
        return

    conn = get_connection()
    if not conn: return
    cur = conn.cursor()


    try:
        cur.execute(sql, params)
        conn.commit()
        print(f"Operation successful: {msg}")
    except Error as e:
        print(f"Update failed: {e}")
        conn.rollback()
    finally:
        cur.close()
        conn.close()

def view_account():
```

```
    print("\n--- View Account ---")
    try:
        acc = int(input("Enter Account Number: ").strip())
    except ValueError: return

    if not verify_pin_input(acc): return

    conn = get_connection()
    if not conn: return
    cur = conn.cursor()

    try:
        cur.execute("SELECT * FROM ACCOUNT WHERE acc_no=%s", (acc,))
        row = cur.fetchone()

        if row:
            print("\n" + "="*35)
            print(f" Account No   : {row[0]}")
            print(f" Name         : {row[1]}")
            print(f" Account Type : {row[6]}")
            print(f" Balance      : ₹{row[5]:,.2f}")
            print(f" Mobile       : {row[3]}")
            print(f" Address      : {row[4]}")
            print("="*35)
        else:
            print("Account not found.")
    except Error as e:
        print(f"Error: {e}")
    finally:
        cur.close()
        conn.close()

def deposit():
    print("\n--- Deposit ---")
    try:
        acc = int(input("Enter Account Number: ").strip())
    except ValueError: return

    amt = input_decimal("Amount to deposit: ")
    if not amt: return

    conn = get_connection()
    if not conn: return
    cur = conn.cursor()

    try:
        cur.execute("SELECT balance FROM ACCOUNT WHERE acc_no=%s FOR UPDATE", (acc,))
```

```
        row = cur.fetchone()
        if not row:
            print("Account not found.")
            return

        new_bal = row[0] + amt
        cur.execute("UPDATE ACCOUNT SET balance=%s WHERE acc_no=%s", (new_bal, acc))
        cur.execute("INSERT INTO TRANSACTION (acc_no, txn_type, amount, remarks) VALUES (%s,
'DEPOSIT', %s, 'Cash Deposit')", (acc, amt))
        conn.commit()
        print(f"Deposit successful. New Balance: ₹{new_bal:,.2f}")
    except Error as e:
        print(f"Error during deposit: {e}")
        conn.rollback()
    finally:
        cur.close()
        conn.close()

def withdraw():
    print("\n--- Withdraw ---")
    try:
        acc = int(input("Enter Account Number: ").strip())
    except ValueError: return

    if not verify_pin_input(acc): return

    amt = input_decimal("Amount to withdraw: ")
    if not amt: return

    conn = get_connection()
    if not conn: return
    cur = conn.cursor()

    try:
        cur.execute("SELECT balance FROM ACCOUNT WHERE acc_no=%s FOR UPDATE", (acc,))
        row = cur.fetchone()
        if not row:
            print("Account not found.")
            return

        curr_bal = row[0]
        if curr_bal < amt:
            print("Insufficient funds.")
            return

        new_bal = curr_bal - amt
        cur.execute("UPDATE ACCOUNT SET balance=%s WHERE acc_no=%s", (new_bal, acc))
```

```
        cur.execute("INSERT INTO TRANSACTION (acc_no, txn_type, amount, remarks) VALUES (%s,
'WITHDRAW', %s, 'Cash Withdrawal')", (acc, amt))
        conn.commit()
        print(f"Withdrawal successful. Remaining: ₹{new_bal:,.2f}")
    except Error as e:
        print(f"Error during withdrawal: {e}")
        conn.rollback()
    finally:
        cur.close()
        conn.close()

def mini_statement():
    print("\n--- Mini Statement ---")
    try:
        acc = int(input("Enter Account Number: ").strip())
    except ValueError: return

    if not verify_pin_input(acc): return

    conn = get_connection()
    if not conn: return
    cur = conn.cursor()

    try:
        cur.execute("""SELECT txn_type, amount, txn_date, remarks
                       FROM TRANSACTION WHERE acc_no=%s
                       ORDER BY txn_date DESC LIMIT 10""", (acc,))
        rows = cur.fetchall()

        if not rows:
            print("No transactions found.")
            return

        print("\n{:<12} {:>12}  {:<20} {:<20}".format("Type", "Amount", "Date", "Remarks"))
        print("-" * 66)
        for r in rows:
            date_str = r[2].strftime("%Y-%m-%d %H:%M")
            print("{:<12} {:>12,.2f}  {:<20} {:<20}".format(r[0], r[1], date_str, r[3] or "-"))
        print("-" * 66)
    except Error as e:
        print(f"Error fetching statement: {e}")
    finally:
        cur.close()
        conn.close()
```

```python
def close_account():
    print("\n--- Close Account ---")
    try:
        acc = int(input("Enter Account Number: ").strip())
    except ValueError: return

    if not verify_pin_input(acc): return

    conn = get_connection()
    if not conn: return
    cur = conn.cursor()

    try:
        # Check balance first
        cur.execute("SELECT balance FROM ACCOUNT WHERE acc_no=%s", (acc,))
        row = cur.fetchone()
        if row and row[0] > 0:
            print(f"Cannot close. Please withdraw remaining ₹{row[0]:,.2f} first.")
            return

        if input("Type 'CLOSE' to confirm: ") == 'CLOSE':
            cur.execute("DELETE FROM ACCOUNT WHERE acc_no=%s", (acc,))
            conn.commit()
            print("Account closed successfully.")
            print("Thanks for banking with us!")
    except Error as e:
        print(f"Error closing account: {e}")
        conn.rollback()
    finally:
        cur.close()
        conn.close()

# --- Main Execution ---
def main():
    execute_db_setup()
    while True:
        print("\n**** BANKING SYSTEM MENU ****")
        print("1. Create Account")
        print("2. View Account")
        print("3. Update Account Details")
        print("4. Deposit")
        print("5. Withdraw")
        print("6. Mini Statement")
        print("7. Close Account")
        print("8. Exit")
```

```
        ch = input("\nEnter Option (1-8): ").strip()

        if ch == '1': create_account()
        elif ch == '2': view_account()
        elif ch == '3': update_account_details()
        elif ch == '4': deposit()
        elif ch == '5': withdraw()
        elif ch == '6': mini_statement()
        elif ch == '7': close_account()
        elif ch == '8':
            print("Thanks for banking with us!")
            break
        else: print("Invalid choice.")

if __name__ == "__main__":
    main()
```

MYSQL

ACCOUNT TABLE:

```
CREATE TABLE ACCOUNT (
  acc_no INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  pin VARCHAR(255) NOT NULL,
  mobile VARCHAR(10),
  address VARCHAR(255),
  balance DECIMAL(12,2) DEFAULT 0.00,
  acc_type VARCHAR(20) DEFAULT 'SAVINGS',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
MariaDB [bank]> desc account;
+------------+---------------+------+-----+-------------------+-------+
| Field      | Type          | Null | Key | Default           | Extra |
+------------+---------------+------+-----+-------------------+-------+
| acc_no     | int(11)       | NO   | PRI | NULL              |       |
| name       | varchar(100)  | NO   |     | NULL              |       |
| pin        | varchar(255)  | NO   |     | NULL              |       |
| mobile     | varchar(15)   | YES  |     | NULL              |       |
| address    | varchar(255)  | YES  |     | NULL              |       |
| balance    | decimal(12,2) | YES  |     | 0.00              |       |
| acc_type   | varchar(20)   | YES  |     | SAVINGS           |       |
| created_at | timestamp     | NO   |     | CURRENT_TIMESTAMP |       |
+------------+---------------+------+-----+-------------------+-------+
8 rows in set (0.01 sec)
```

TRANSACTION TABLE:

```
CREATE TABLE TRANSACTION (
  txn_id INT AUTO_INCREMENT PRIMARY KEY,
  acc_no INT NOT NULL,
  txn_type VARCHAR(20) NOT NULL,     -- DEPOSIT, WITHDRAW
  amount DECIMAL(12,2) NOT NULL,
  txn_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  remarks VARCHAR(255),
  FOREIGN KEY (acc_no) REFERENCES ACCOUNT(acc_no) ON DELETE CASCADE
);
```

```
MariaDB [bank]> desc transaction;
+----------+---------------+------+-----+-------------------+----------------+
| Field    | Type          | Null | Key | Default           | Extra          |
+----------+---------------+------+-----+-------------------+----------------+
| txn_id   | int(11)       | NO   | PRI | NULL              | auto_increment |
| acc_no   | int(11)       | NO   | MUL | NULL              |                |
| txn_type | varchar(20)   | NO   |     | NULL              |                |
| amount   | decimal(12,2) | NO   |     | NULL              |                |
| txn_date | timestamp     | NO   |     | CURRENT_TIMESTAMP |                |
| remarks  | varchar(255)  | YES  |     | NULL              |                |
+----------+---------------+------+-----+-------------------+----------------+
6 rows in set (0.04 sec)
```

OUTPUT

```
**** BANKING SYSTEM MENU ****
1. Create Account
2. View Account
3. Update Account Details
4. Deposit
5. Withdraw
6. Mini Statement
7. Close Account
8. Exit

Enter Option (1-8): 1

--- Open New Account ---
Enter Account Number: 1001
Enter Full Name: Rohan Sharma
Select Account Type:
1. Savings
2. Current
Choice (1/2): 1
Set 4-digit PIN: 1234
Mobile: 9876543210
Address: Delhi, India
Initial Deposit: 4500
Account created successfully. Account Type: Savings.
```

```
**** BANKING SYSTEM MENU ****
1. Create Account
2. View Account
3. Update Account Details
4. Deposit
5. Withdraw
6. Mini Statement
7. Close Account
8. Exit

Enter Option (1-8): 2

--- View Account ---
Enter Account Number: 1002
Enter PIN to confirm: 5678

==================================
 Account No   : 1002
 Name         : Karan Singh
 Account Type : SAVINGS
 Balance      : ₹25,000.00
 Mobile       : 9123456780
 Address      : Kolkata, India
==================================
```

```
Enter Option (1-8): 3

--- Update Account Details ---
Enter Account Number: 1002
Enter PIN to confirm: 5678

What would you like to update?
1. Mobile Number
2. Address
3. Change PIN
4. Cancel
Enter choice (1-4): 2
Enter new Address: Patna , Bihar , India
Operation successful: Address updated.
```

```
Enter Option (1-8): 4

--- Deposit ---
Enter Account Number: 1002
Amount to deposit: 5000
Deposit successful. New Balance: ₹30,000.00
```

```
Enter Option (1-8): 5

--- Withdraw ---
Enter Account Number: 1002
Enter PIN to confirm: 5678
Amount to withdraw: 1800
Withdrawal successful. Remaining: ₹28,200.00
```

```
Enter Option (1-8): 6

--- Mini Statement ---
Enter Account Number: 1002
Enter PIN to confirm: 5678

Type               Amount  Date                Remarks
-----------------------------------------------------------------
WITHDRAW         1,800.00  2025-11-26 20:23    Cash Withdrawal
WITHDRAW           500.00  2025-11-26 20:22    Cash Withdrawal
DEPOSIT            500.00  2025-11-26 20:22    Cash Deposit
DEPOSIT          5,000.00  2025-11-26 20:22    Cash Deposit
-----------------------------------------------------------------
```

```
Enter Option (1-8): 7

--- Close Account ---
Enter Account Number: 1001
Enter PIN to confirm: 1234
Cannot close. Please withdraw remaining ₹4,500.00 first.
```

```
--- Withdraw ---
Enter Account Number: 1001
Enter PIN to confirm: 1234
Amount to withdraw: 4500
Withdrawal successful. Remaining: ₹0.00

**** BANKING SYSTEM MENU ****
1. Create Account
2. View Account
3. Update Account Details
4. Deposit
5. Withdraw
6. Mini Statement
7. Close Account
8. Exit

Enter Option (1-8): 7

--- Close Account ---
Enter Account Number: 1001
Enter PIN to confirm: 1234
Type 'CLOSE' to confirm: CLOSE
Account closed successfully.
Thanks for banking with us!
```

```
**** BANKING SYSTEM MENU ****
1. Create Account
2. View Account
3. Update Account Details
4. Deposit
5. Withdraw
6. Mini Statement
7. Close Account
8. Exit

Enter Option (1-8): 8
Thanks for banking with us!
```

BIBLIOGRAPHY

The following resources were consulted during the development of this project:

1. Python Software Foundation. Python 3.x Documentation. [https://docs.python.org/]
2. MySQL Documentation. MySQL Reference Manual. [https://dev.mysql.com/doc/]
3. XAMPP Apache Friends. MySQL & phpMyAdmin Usage Guide. [https://www.apachefriends.org/]
4. W3Schools. Python and SQL Tutorials. [https://www.w3schools.com/]
5. GeeksforGeeks. Python Database Connectivity (MySQL) Tutorials. [https://www.geeksforgeeks.org/]
6. Computer Science with Python - Class XI & XII by Sumita Arora